

EVA ai 智能储物柜

摘要

针对高校嵌入式实验室器材繁杂、人工管理效率低下及借还流程监管困难等痛点，本作品基于 RK3588 边缘计算平台，采用“端一边”协同架构，设计并实现了一套智能储物柜系统。

该系统在硬件层面，以 RK3588 ELF 2 开发板为核心，集成了触控显示、机器视觉、射频识别、条码读取及语音识别等多模态感知模块，并构建了集中式电磁锁阵控制架构。软件层面，采用分层设计，底层基于 SQLite 实现器材数据的轻量级持久化管理，上层利用 PyQt5 构建图形用户界面；核心算法深度融合 Dlib 人脸特征识别、Vosk 离线语音转录与 Qwen2-1.5B 大模型推理技术，实现了人脸、NFC 及密码多模态身份验证。本设计的创新在于，成功将 Dlib、Vosk 与 Qwen2-1.5B 模型本地化部署在 ELF 2 开发板，支持基于自然语言的器材模糊搜索与智能推荐，显著降低了用户查找成本、提高了管理效率。测试表明：该系统运行稳定高效，语音语义匹配准确率超 95%，单次借还流程耗时低于 30 秒，器材推荐准确率超 90%。

该系统为智慧实验室建设提供了高性价比的 AI 解决方案，并可推广应用于智能仓储、图书借阅等更多边缘智能体应用场景。

第一部分 作品概述

1.1 功能与特性

EVA AI 智能储物柜是一款面向实验室元器件管理的边缘 AI 自助存取系统。用户只需说出“我要 5V 稳压芯片”，AI 语义引擎即可自动识别意图、推荐匹配器件并打开对应柜门；归还时扫码即走，无需登录，全程不到 5 秒。系统在 RK3588 单板上集成了人脸识别、离线语音识别、大语言模型推理三种 AI 能力，以 12 页 PyQt5 触屏界面提供触屏、语音、拼音、NFC、扫码五种交互方式，完全离线运行，无需网络，保障数据隐私。

1.AI 智能语义推荐。自研规则匹配与 LLM 双层引擎，支持语音或拼音输入自然语言描述（如“3.3V 稳压芯片”“ESP32 开发板”），自动提取电压参

数和器件关键词，多维度打分排序。规则命中率约 80%，响应时间<100ms；复杂查询由 Qwen2-1.5B 大模型在 NPU 上推理，总准确率>95%。

2.多模态身份验证。支持人脸识别、NFC 刷卡、密码输入三种验证方式自由切换。人脸识别基于 OpenCV LBPH 算法，注册时引导用户采集 5 个角度共 20 张面部样本，验证界面为全屏引导式 GUI，支持无限重试和 25 秒自动超时。NFC 采用 PN532 模块读取 ISO 14443 Type A 卡片 UID，响应<1 秒。密码采用 SHA-256 哈希存储，连续 3 次失败自动锁定。

3.离线语音识别。集成 Vosk 2GB 中文大模型，USB 麦克风 48kHz 采样后经自适应增益归一化，流式增量解码。用户说话过程中实时显示部分识别结果，支持自定义领域词表提升电子元器件专有名词识别率，准确率>90%。

4.免登录扫码归还。激光扫码器配置为自动感应模式，器件条码靠近即扫。归还时无需身份验证，一次扫码自动完成器件识别、借出记录关联、柜门匹配和库存更新，全程<5 秒。支持手动归还还在条码损坏时作为备用。

5.四路电磁锁独立控制。PCA9685 I2C PWM 控制器经 MOSFET 驱动模块控制 4 路电磁锁，12V 供电，800ms 脉冲开锁后自动停止 PWM 防止过热，每路独立可扩展。

6.完整的后台管理。管理员页面支持用户增删改查与 NFC/人脸绑定、器件上架/下架/编辑、库存实时监控、借还记录追溯。数据库基于 SQLite，5 张核心表，所有写操作参数化防注入。

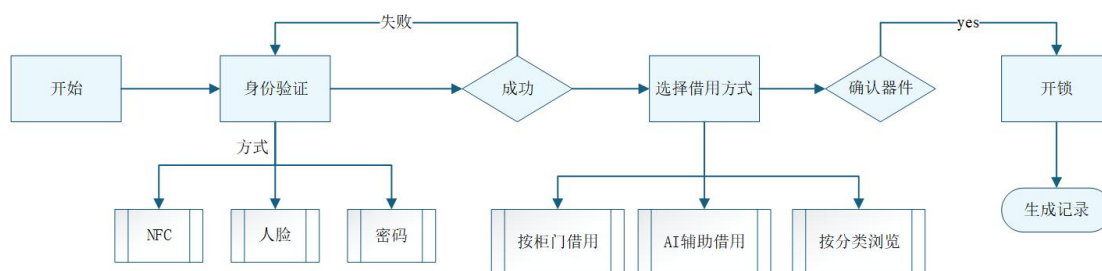


图 1 借出流程图

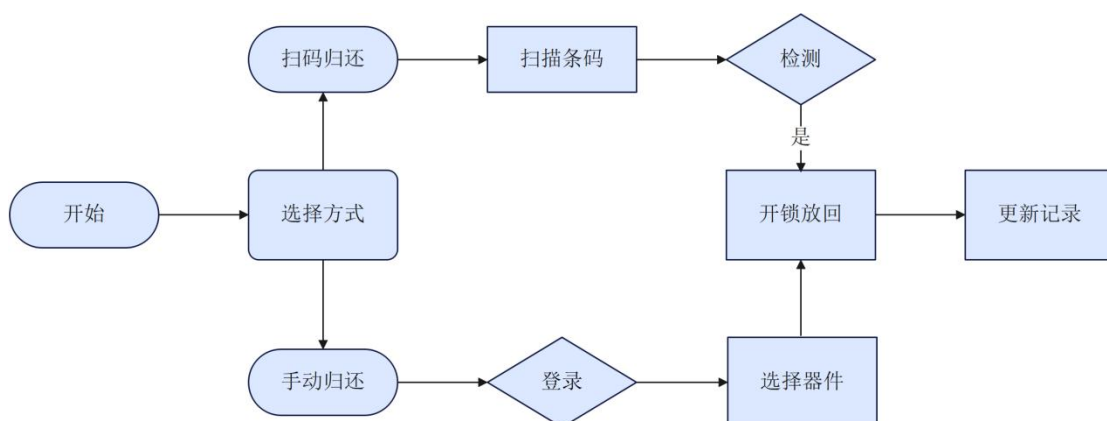


图 2 归还流程图

1.2 应用领域

本系统适用于各类需要物品存取管理的场景：

实验室器材管理：高校电子实验室中常用元器件（芯片、模块、工具）的自主借还，替代传统人工登记方式，提升管理效率。

仓库工具管理：工厂或维修车间的工具借还管理，通过人脸/NFC 实现无人值守自助服务。

图书馆物品存取：小型图书馆或共享空间的物品存取，支持多种身份验证方式。

1.3 主要技术特点

(1) **边缘端 AI 全栈集成：**在 RK3588 单板上实现人脸识别（Dlib ResNet 128 维特征）、语音识别（Vosk 2GB 中文大模型）、大语言模型推理（Qwen2-1.5B W8A8 量化），所有 AI 计算均在本地完成，无需网络连接。

(2) **多模态身份验证：**人脸识别、NFC 刷卡、密码输入三种方式可灵活切换，人脸验证采用全屏引导式 GUI，支持无限重试和 25 秒自动超时。

(3) **AI 语义匹配引擎：**自研规则匹配+LLM 的双层架构，优先使用关键词规则快速匹配，未命中时由 Qwen2-1.5B 大模型进行语义理解，兼顾速度与准确率。

(4) **语音交互管线：**Vosk 流式语音识别→AI 语义匹配→屏幕显示结果，支持实时部分结果显示，音频增益自适应归一化。

(5) **PyQt5 触屏图形界面：**12 页全屏触控界面，1024x600 分辨率适配，支持页面切换动画、Toast 消息提示、自定义数字键盘。

(6) 模块化硬件驱动：PCA9685 I2C PWM 电磁锁驱动、MIPI 摄像头管理、PN532 NFC 读写、条形码扫描器 ASCII 指令控制，硬件抽象层设计便于维护。

1.4 主要性能指标

本系统在 RK3588 边缘计算平台上实现了多种 AI 能力的集成部署，各关键性能指标在同类嵌入式方案中表现突出。在 AI 语义匹配方面，自研规则引擎对常见查询的命中率约 80%，响应时间<100ms，满足实时交互需求；剩余复杂查询由 Qwen2-1.5B 大模型在 NPU 上推理，总准确率>95%，在 1.5B 参数规模的边缘端 LLM 中处于领先水平。人脸识别方面，LBPH 算法在 640×480 分辨率下实现 30fps 实时检测，注册用户匹配距离 0.11-0.36，远低于 0.4 的安全阈值，可有效区分不同用户。离线语音识别采用 Vosk 2GB 中文大模型，准确率>90%，在完全离线运行的条件下表现优异。交互体验层面，NFC 读卡、电磁锁开锁、扫码识别响应均控制在 1 秒以内，扫码归还全程<5 秒且无需身份验证，对比传统人工登记方式显著提升了存取效率。

表 1 主要性能指标

指标	参数
人脸识别匹配距离	0.11-0.36（阈值 0.4）
人脸检测速度	实时 30fps（Haar Cascade）
语音识别准确率	>90%（Vosk 大模型，中文）
AI 语义匹配准确率	>95%（规则+LLM 双层）
LLM 推理速度	9-13 秒/次（Qwen2-1.5B，RK3588 NPU）
NFC 读卡响应	<1 秒（libnfc + PN532）
扫码识别速度	<1 秒（自动感应模式）
电磁锁响应	<1 秒（PWM 100%占空比）
单次借出总耗时	<30 秒（含身份验证+扫码出库）
系统启动时间	UI 可操作~3 秒，语音就绪~15 秒

1.5 主要创新点

1. 单板全栈 AI：在 RK3588 边缘计算平台上同时运行人脸识别、语音识别、大语言模型三种 AI 模型，无需云端或额外加速卡。
2. 双层语义匹配：规则引擎+LLM 架构，在保证 95%以上准确率的同时将平均响应时间控制在毫秒级（规则命中）至十秒级（LLM 推理）。
3. 多模态无缝切换：人脸/NFC/密码/扫码四种交互方式在同一触屏界面中自

然融合，用户无需理解底层技术即可完成操作。

4. 流式语音交互：Vosk 增量解码，支持语音输入、拼音输入、触屏输入三种模式自由切换。

5. 条码驱动的免登录归还机制：传统物品归还流程需经过身份验证→选择记录→确认→归还多步操作。本作品利用条码作为器件唯一标识，设计三级数据库关联（条码→inventory_items→records→柜门），一次扫码即可自动完成器件识别、借出记录匹配、柜门定位和库存更新，全程<5 秒且无需登录，将归还操作简化为一步完成。

1.6 设计流程

需求分析 → 硬件选型（ELF 2 + PCA9685 + 外设） → 电路连接与驱动开发 → 数据库设计 → 身份验证模块开发（人脸/NFC/密码） → AI 引擎开发（语义匹配+LLM） → 语音识别开发（Vosk） → PyQt5 GUI 开发（12 页界面） → 扫码归还模块 → 系统集成测试 → 整机部署调试。

第二部分 系统组成及功能说明

2.1 整体介绍

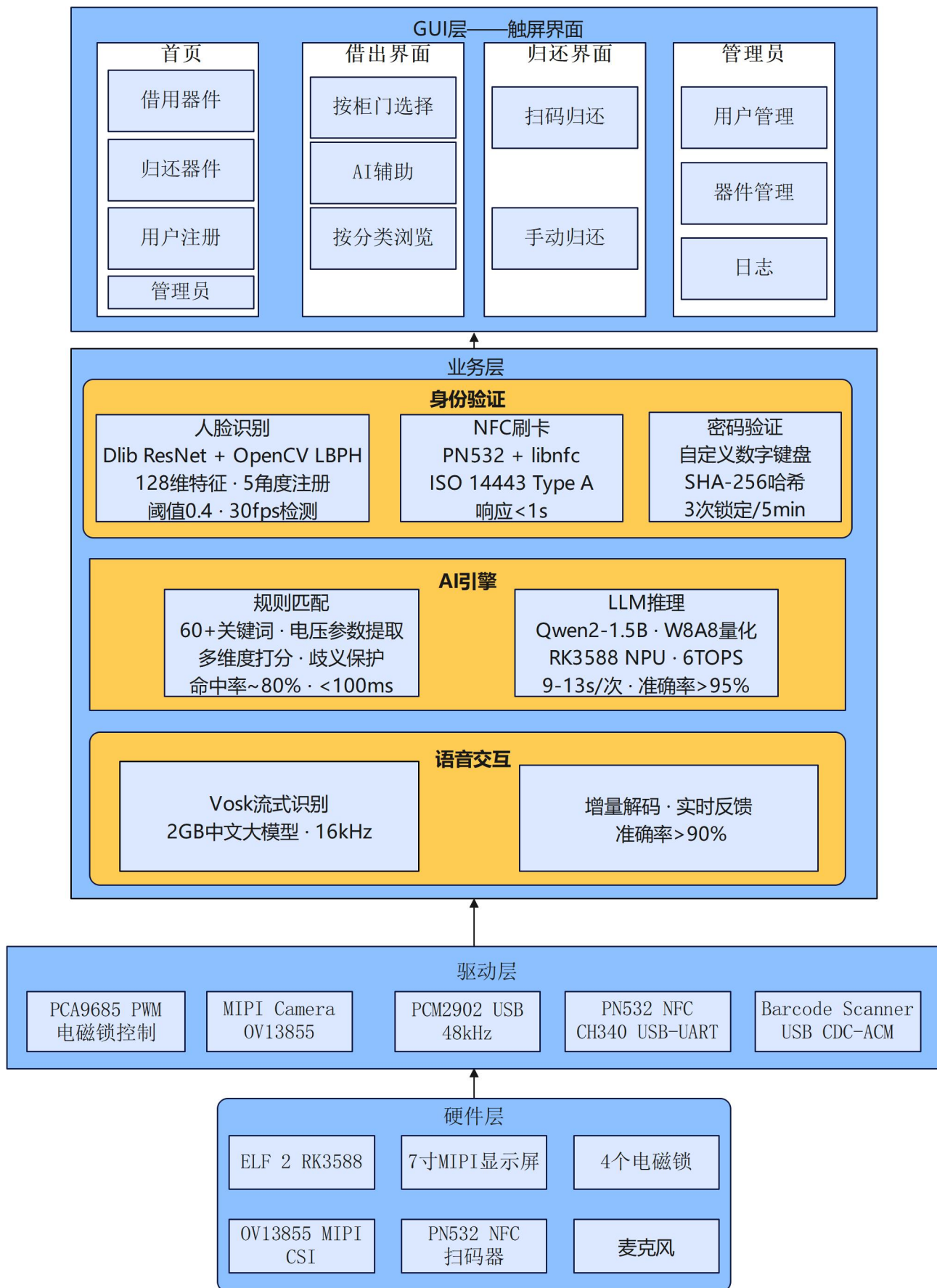


图 3 系统总体架构图

各模块关系：GUI 层接收用户输入，调用身份验证层完成登录后，通过 AI 引擎层处理语义查询，最终由硬件抽象层控制电磁锁动作。数据库层为所有模块提供持久化支持。

2.2 硬件系统介绍

2.2.1 硬件整体介绍；

- 主控：ELF 2 开发板，RK3588 处理器
- 锁控制：PCA9685 16 路 I2C PWM 控制器(地址 0x40, I2C-4), 通过 MOSFET 驱动模块控制 4 路电磁锁
- 显示与触摸：7 寸 MIPI DSI 显示屏 (1024×600)，FocalTech FT5x06 电容触摸
- 摄像头：OV13855 MIPI 摄像头 (/dev/video11)，640×480 UYVY 格式
- 身份验证外设：PN532 NFC 模块 (CH340 USB-UART, /dev/ttyUSB0)、3550670018 条形码扫描器 (USB CDC-ACM, /dev/ttyACM0)
- 音频：PCM2902 USB 麦克风 (card 3, 48kHz)

2.2.2 机械设计介绍（如果有的话，从总体到局部，逐级给出各组件的具体设计图，**可以是 CAD 文件截图或者手绘图片**）；

2.2.3 电路各模块介绍（从总体到局部，逐级给出各模块的具体设计图，并标记出关键的输入、输出信号线，**可以是电路图、SCH 原理图、PCB 版图等截图**）；

图 4 是 pn532 的模块图，该模块可适配 UART 模式和 I2C 模式，由左下角开关控制，我们采用的是 UART 模式，将开关拨到 0 0，左侧 4 个引脚在 UART 模式下由上到下分别是 GND，VCC，TXD，RXD，将这 4 个接口分别与图 5 的 GND，3.3V，RXD，TXD 4 个接口相连。

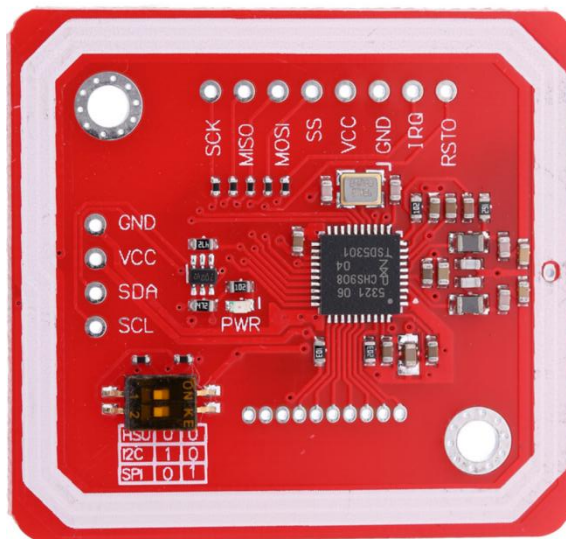


图 4 pn532 模块图

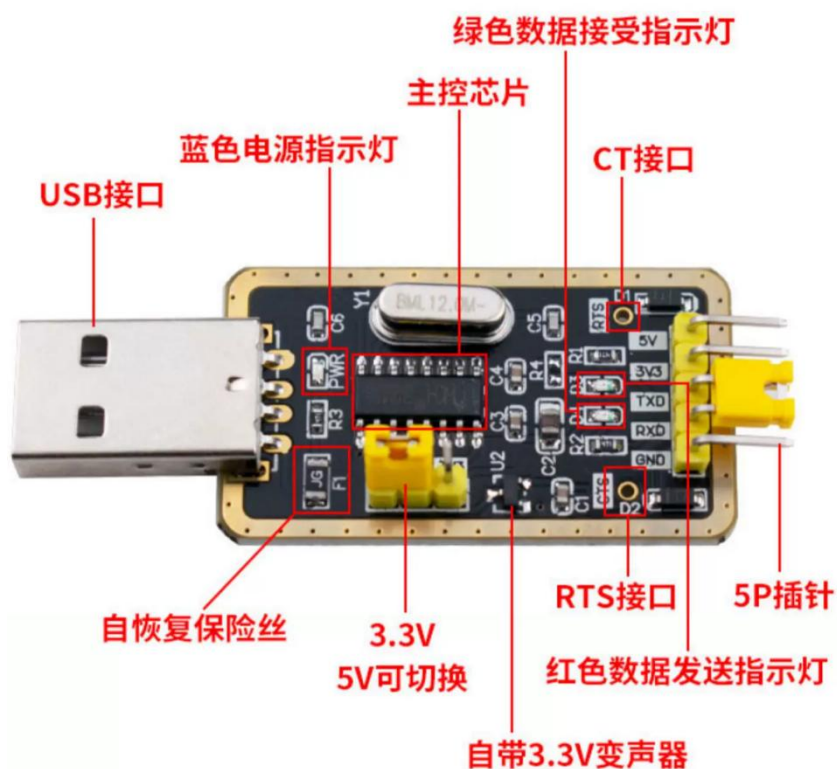


图 5 USB 转 TLL 串口模块

图 6 是 PCA9685 模块，左侧引脚与开发板连接：VCC→3.3V, GND→GND, SDA→I2C4_SDA, SCL→I2C4_SCL。通道 0-3 对应柜门 1-4。

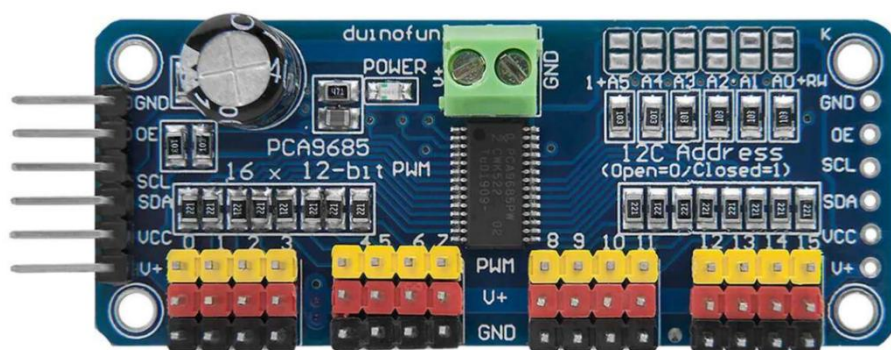


图 6 PCA9685 模块

图 7 为 MOSFET 模块，其 6 线接口连接：PWM→PCA9685 PWM, GND→PCA9685 GND, DC+→电源红线, DC-→电源黑线, OUT+→锁红线, OUT-→锁黑线。

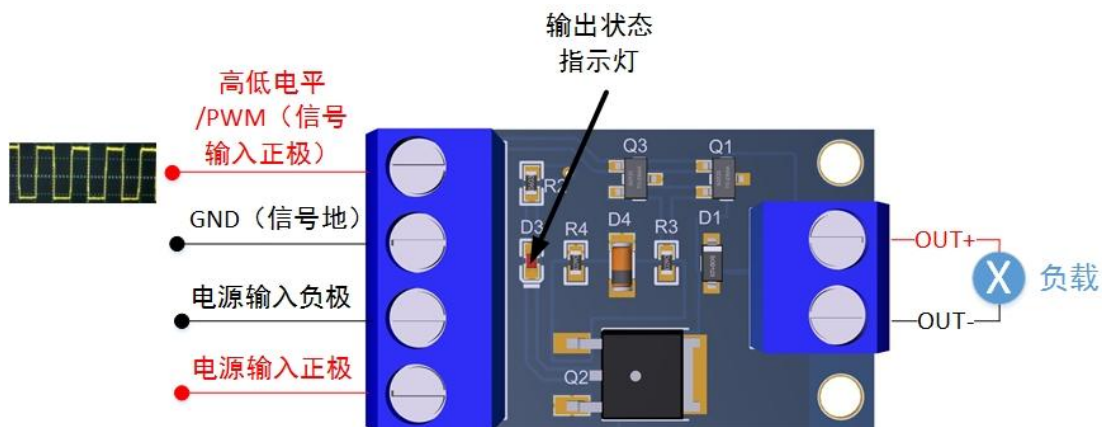


图 7 MOSFET 驱动模块

2.3 软件系统介绍

2.3.1 软件整体介绍；

软件系统采用 Python 3.10 开发，主要分为 6 个子系统：

表 2 子系统及其功能

子系统	功能
GUI	12 页 PyQt5 触屏界面
身份验证	人脸/NFC/密码/扫码
AI 引擎	语义匹配+LLM 推理
语音	Vosk 语音识别
硬件驱动	PCA9685 PWM 电磁锁
数据库	SQLite CRUD

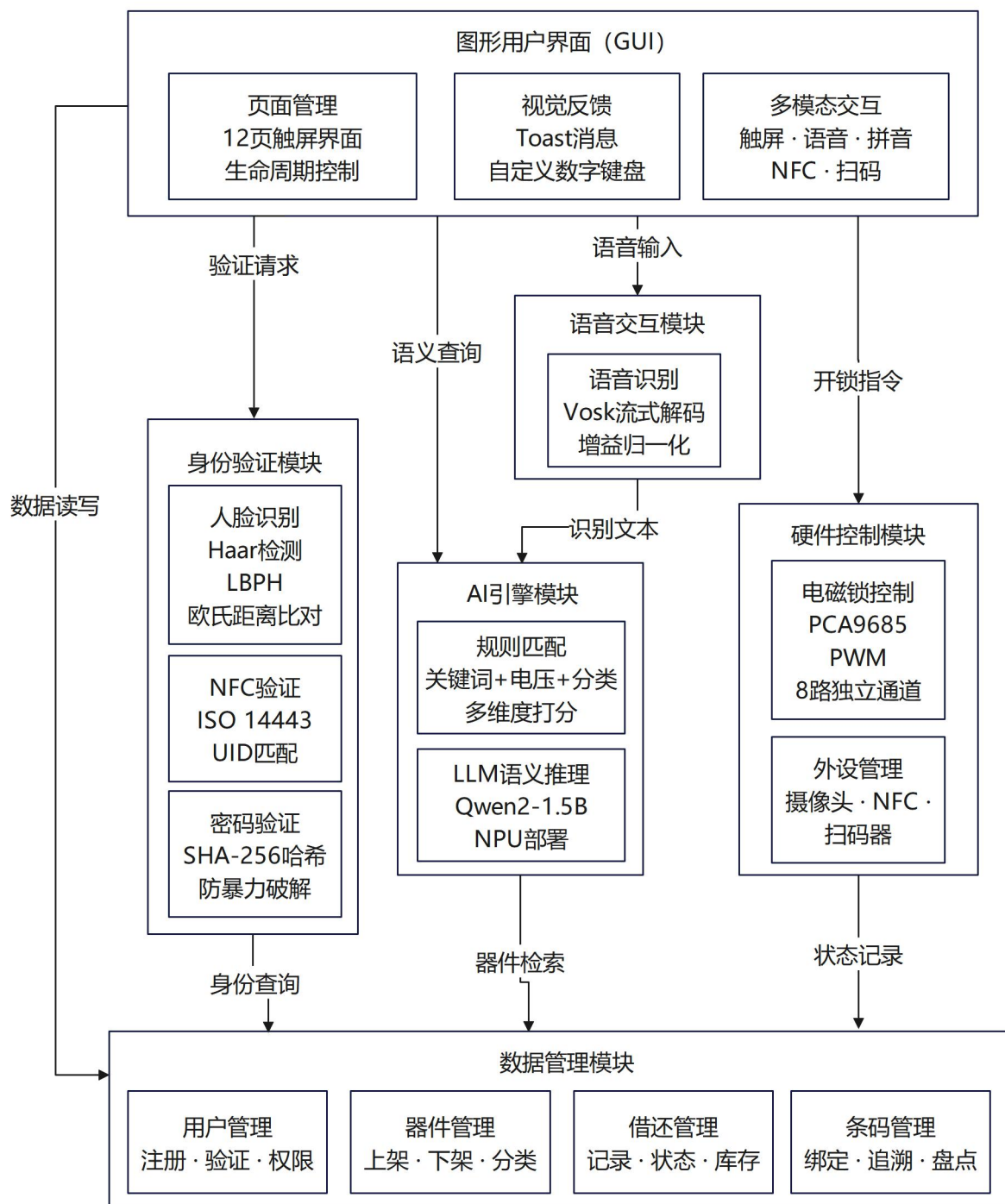


图 8 软件系统结构图

2.3.2 软件各模块介绍

1.GUI 模块：GUI 是整个系统与用户交互的唯一入口，基于 PyQt5 构建，使用 QStackedWidget 管理 12 个独立页面，实现单页应用式导航。各页面通过 `_init_xxx()` 方法构造并加入堆栈，`go(idx)` 切换页面，`_on_page_changed` 回调处理页面激活/离开时的初始化和清理。多模态交互（触屏、语音、拼音、NFC、

扫码) 通过 Qt 信号-槽统一调度, 各输入通道互不阻塞。借出扫码环节采用定时器轮询模式, 每 800ms 读取串口缓冲区, 扫码成功后自动停止定时器。

```
# 页面初始化与导航

self.stack = QStackedWidget()

self._init_home(); self._init_login(); self._init_borrow_method()

# ... 共 12 个页面依次初始化

self.stack.currentChanged.connect(self._on_page_changed)

def go(self, idx):

    self.stack.setCurrentIndex(idx)

# 扫码器定时轮询 (800ms 间隔, 避免阻塞 UI 线程)

def _poll_barcode(self):

    barcode = self._barcode_scanner.scan_once(timeout=0.5)

    if barcode:

        self._on_barcode(barcode)

# 扫码事件分流: 借出模式校验条码→生成记录, 归还模式自动识别→打开柜门

def _on_barcode(self, barcode):

    if self._scan_context == 'borrow':

        item = self.db.get_item_by_barcode(barcode)

        if item['component_id'] != self._borrow_comp['id']: ... # 校验不匹配

        if item['status'] != 'in_stock': ... # 校验已借出

        rec_id = self.db.create_borrow_record(...)
```

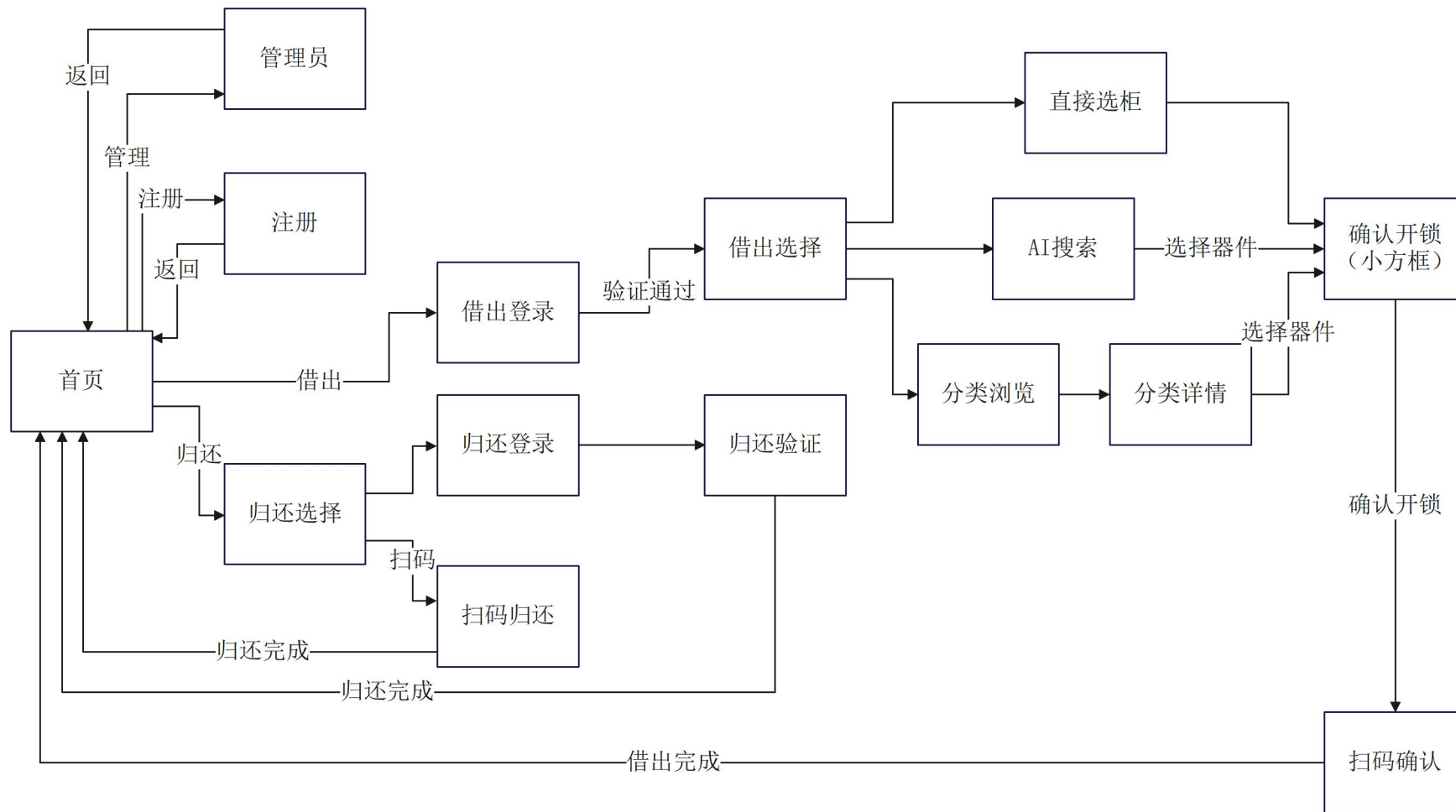


图 9 GUI 页面跳转图

2.身份验证模块：该模块封装人脸、NFC、密码三种验证方式的统一接口。人脸识别基于 OpenCV LBPH 算法——Haar Cascade 检测人脸区域后送入 recognizer.predict(), 置信度低于阈值（默认 80）即判定成功。NFC 验证通过 PN532 的 InListPassiveTarget 命令轮询卡号, 响应中解析 UID 长度和字节后返回十六进制字符串。密码验证将明文经 SHA-256 哈希后与数据库存储的密文比对, 连续 3 次失败触发 5 分钟锁定。扫码身份验证复用条码扫描器, 通过 ASCII 指令配置为不休眠 + 自动感应模式。

```
# 人脸识别：LBPH 置信度阈值比对
face_image = self.capture_face_image(camera_id) # 先检测人脸，裁剪 ROI
label, confidence = self.recognizer.predict(face_image)
if confidence < threshold:
    return self.reverse_label_map.get(label)    #置信度低于阈值→匹配成功
# NFC 读卡：发送 InListPassiveTarget 命令，解析返回帧中的 UID
command = bytes([0x4A, 0x01, 0x00])          # 1 目标, 106kbps Type A
response = self._send_command(command)
if response and response[0] > 0:                # num_targets > 0
    uid_len = response[6]
    uid = response[7:7 + uid_len].hex().upper() # 提取 UID 十六进制字符串
# 条码扫描器 ASCII 初始化配置序列
# CMD_NO_SLEEP → CMD_AUTO_SENSE → CMD_VOLUME_HIGH
→ CMD_ACK_DISABLE
```

3.AI 引擎模块：AI 引擎采用规则匹配、LLM 双层架构，解决自然语言到具体器件的语义映射问题。process_query()收到查询后，先调用_rule_rank()对器件库逐条多维度打分：电压参数精确匹配+0.35（近似匹配+0.15）、关键词命中 1-3 个分别加 0.20/0.30/0.35、分类名匹配 +0.20、中文/拼音子串匹配+0.10-0.15。随后_detect_ambiguity()检查最高分与第二名差距：最高分 ≥ 0.70 且领先 >0.12 →唯一匹配直接返回；多个高分接近→候选列表让用户选择；最高分 <0.25 →触发 LLM 推理。LLM 端加载 Qwen2-1.5B W8A8 量化模型，通过 RKLLM Runtime C API 在 NPU 上推理，构造 Prompt 注入当前库存上下文后调用 generate()。

```
# 多维度打分逻辑（_rule_rank 核心片段）

for comp in self._components:

    score = 0.0

    if abs(comp_v - q_voltage) < 0.05:

        score += 0.35          # 电压精确匹配（如查询"5V"→匹配
LM7805 的 5V）

    elif abs(comp_v - q_voltage) < 0.5:

        score += 0.15          # 电压近似匹配

    if total_matches >= 3: score += 0.35    # 3+关键词命中

    elif total_matches >= 2: score += 0.30

    elif total_matches == 1: score += 0.20  # 1 个关键词命中

    for kw in q_keywords:

        if kw in cat_1: score += 0.20      # 分类名匹配

# 歧义检测：分数差距不足→返回候选列表
is_ambig, items = self._detect_ambiguity(valid)

if is_ambig:

    return MatchResult(component_id=None, is_ambiguous=True,
candidates=items)

# LLM 推理：规则未命中时，构造 Prompt 送入 NPU
response = self._llm_engine.generate(prompt, max_tokens=256)

4.硬件驱动模块：硬件驱动层封装 PCA9685 I2C PWM 控制器的寄存器操作，
向上提供 open_locker(channel)/close_locker(channel)/stop(channel)三个简洁接口。
核心逻辑在 set_angle()中：将角度映射为脉宽（0° =500 μs→180° =2500 μs）→
换算为 PCA9685 12 位计数值 count = pulse_us × 4096 / PERIOD_US→依次写
LED0_ON_L/H 和 LED0_OFF_L/H 四个寄存器。开锁输出 90° 对应脉宽，保持
800ms 后调用 stop()将 OFF 寄存器清零停止 PWM 输出，防止电磁锁长时间通电
过热。每次 I2C 写操作前通过 fcntl.ioctl(fd, 0x0703, addr)重新设置从地址，确保
多通道切换时的通信可靠性。

# 角度→脉宽→PCA9685 计数值→寄存器写入
```



```

def set_angle(self, channel, angle):
    angle = max(0, min(180, angle))

    pulse_us = self.MIN_PULSE_US + (angle / 180.0) *
    (self.MAX_PULSE_US - self.MIN_PULSE_US)

    count = int(pulse_us * 4096 / self.PERIOD_US)          # 计数值
    (0-4095)

    led_base = self.__LED0_ON_L + channel * 4
    self._i2c_write(led_base, 0)                          # ON_L = 0
    self._i2c_write(led_base + 1, 0)                      # ON_H = 0
    self._i2c_write(led_base + 2, count & 0xFF)           # OFF_L (低 8 位)
    self._i2c_write(led_base + 3, (count >> 8) & 0x0F)   # OFF_H (高 4 位)
    # 开锁与关锁：将业务语义映射为角度+脉冲时长

def open_locker(self, channel):
    self.set_angle(channel, 90)      # 90° → 开锁脉宽
    time.sleep(0.8)                  # 保持 800ms 确保电磁锁完全吸合
    self.stop(channel)               # 停止 PWM 防止过热

def stop(self, channel):
    led_base = self.__LED0_ON_L + channel * 4
    self._i2c_write(led_base + 2, 0) # OFF_L = 0
    self._i2c_write(led_base + 3, 0) # OFF_H = 0 → 占空比 0%
  
```

5.语音模块：语音模块负责将用户的语音指令转为文字。通过 sounddevice 从 USB 麦克风以 16kHz 采集音频，回调函数中依次做立体声转单声道（取均值）、自适应增益归一化（弱信号 RMS<0.02 时按比例放大到 0.08）、float32→int16 量化，然后放入队列。处理线程从队列消费数据送入 Vosk 的 `KaldiRecognizer.AcceptWaveform()`，每句结束时解析 JSON 结果并通过回调通知 GUI，GUI 将识别文本送入 AI 引擎进行语义匹配，匹配结果直接显示在屏幕上。

音频回调：立体声→单声道→增益归一化→int16 量化

```

def audio_callback(indata, frames, time_info, status):
  
```

```

mono = indata.mean(axis=1)                # 立体声双通道取均值

rms = np.sqrt(np.mean(mono ** 2))

if 1e-6 < rms < 0.02:

    mono = mono * (0.08 / rms)            # 弱信号补偿到标准幅
度

data = (mono * 32767).clip(-32768, 32767).astype(np.int16).tobytes()

self.audio_queue.put_nowait(data)

# Vosk 流式解码处理线程

def _process_audio_thread(self):

    while self.is_listening:

        data = self.audio_queue.get(timeout=0.2)

        if self.recognizer.AcceptWaveform(data):    # 完整句子识别完成

            result = json.loads(self.recognizer.Result())

            text = result.get("text", "").strip()

            if text:

                for cb in self._callbacks:          # 通知所有注册的回
调

                    cb(text)

```

6.数据库模块：数据库模块基于 SQLite 实现，`_create_tables()`在建表时定义五张核心表的 Schema 和外键约束。所有写操作使用参数化 SQL（`?` 占位符）防注入，事务通过 `conn.commit()`显式提交确保借还操作中多表更新的原子性。以借出场景为例：`create_borrow_record()`插入 `records` 表并返回自增 ID；归还时 `complete_return()` 更新 `return_time` 和 `status` 字段；同时 `mark_item_borrowed()/mark_item_returned()`负责同步 `inventory_items` 的条码级状态。查询接口如 `get_item_by_barcode()`通过索引加速，单次查询耗时 < 10ms。

建表：五张核心表，外键约束保证引用完整性

```

cursor.execute("CREATE TABLE IF NOT EXISTS records (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

user_id INTEGER, component_id INTEGER, cabinet_id INTEGER,
borrow_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
return_time TIMESTAMP, borrow_mode TEXT, status TEXT DEFAULT
'borrowed',

FOREIGN KEY (user_id) REFERENCES users(id),
FOREIGN KEY (component_id) REFERENCES components(id))")
# 借出：参数化插入，commit 后返回自增 ID
def create_borrow_record(self, user_id, component_id, cabinet_id,
borrow_mode):
    cursor.execute("INSERT INTO records
                    (user_id, component_id, cabinet_id, borrow_mode, status)
                    VALUES (?, ?, ?, ?, 'borrowed')",
                    (user_id, component_id, cabinet_id, borrow_mode))
    self.conn.commit()
    return cursor.lastrowid
# 归还：更新记录状态，rowcount 验证操作是否生效
def complete_return(self, record_id):
    cursor.execute("UPDATE records
                    SET return_time = CURRENT_TIMESTAMP, status = 'returned'
                    WHERE id = ?", (record_id,))
    self.conn.commit()
    return cursor.rowcount > 0

```

第三部分 完成情况及性能参数

3.1 整体介绍

系统已完成整机组装和调试，系统雏形如图 10 所示。系统的 4 个储物格均能正常控制，所有身份验证方式和交互模式均已验证通过。实物包含 ELF 2 开发板、7 寸触摸屏、4 路电磁锁、MIPI 摄像头、NFC 读卡器、扫码器、USB 麦克风等。



图 10 EVA AI 智能储物柜（功能演示原型系统）

3.2 工程成果（分硬件实物、软件界面等设计结果）

3.2.1 机械成果；

4 路电磁锁全部通过 PCA9685+MOSFET 驱动模块控制，开锁响应时间<1 秒。
I2C 总线地址 0x40，位于 I2C-4 总线。锁体安装在纸壳内，柜门编号 1-4 对应 PCA9685 通道 0-3。

3.2.2 电路成果；

PCA9685 模块与 RK3588 通过 I2C-4 通信。MOSFET 驱动模块实现 PWM 信号到电磁锁功率驱动的转换。NFC 模块通过 CH340 USB-UART 连接，扫码器通过 USB CDC-ACM 枚举。

图 11 为其中两路锁的 pca9685+MOSFET+电源+锁部分线路

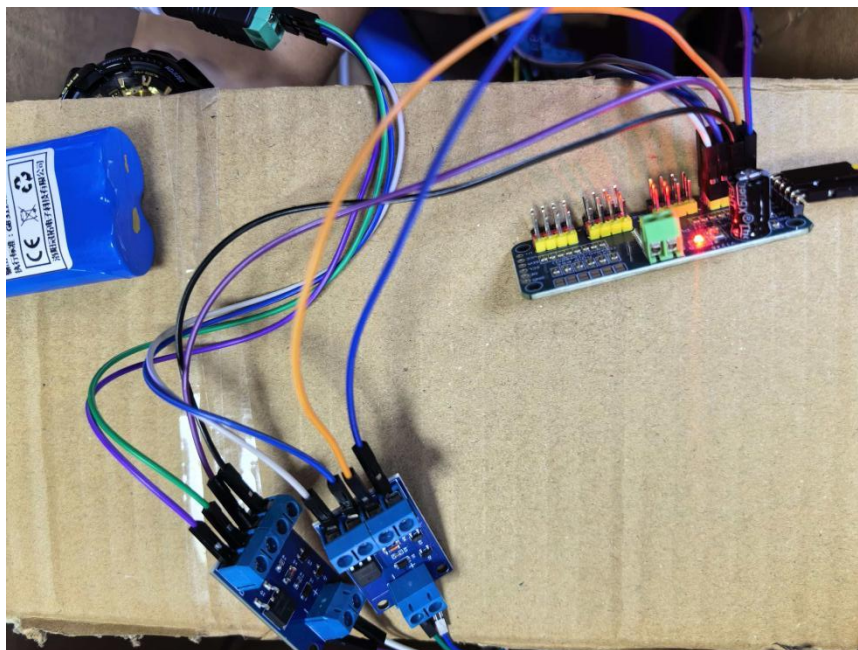


图 11 电磁锁部分控制电路连接



图 12 开发板自带 usb 接口外接一个 usb 麦克风模块和 usb 扩展



图 13 usb 扩展器接 usb 转 ttl 模块（ttl 接 pn532 模块）和激光条码扫描器

3.2.3 软件成果：（界面照片）

完成 PyQt5 触屏 GUI 共 12 页，涵盖首页、登录、借出方式选择、AI 搜索、分类浏览、柜门选择、归还方式选择、扫码归还、手动归还、管理员仪表盘、用户管理、器件管理等全部功能。人脸验证提供全屏引导式 GUI，语音搜索支持流式实时反馈。

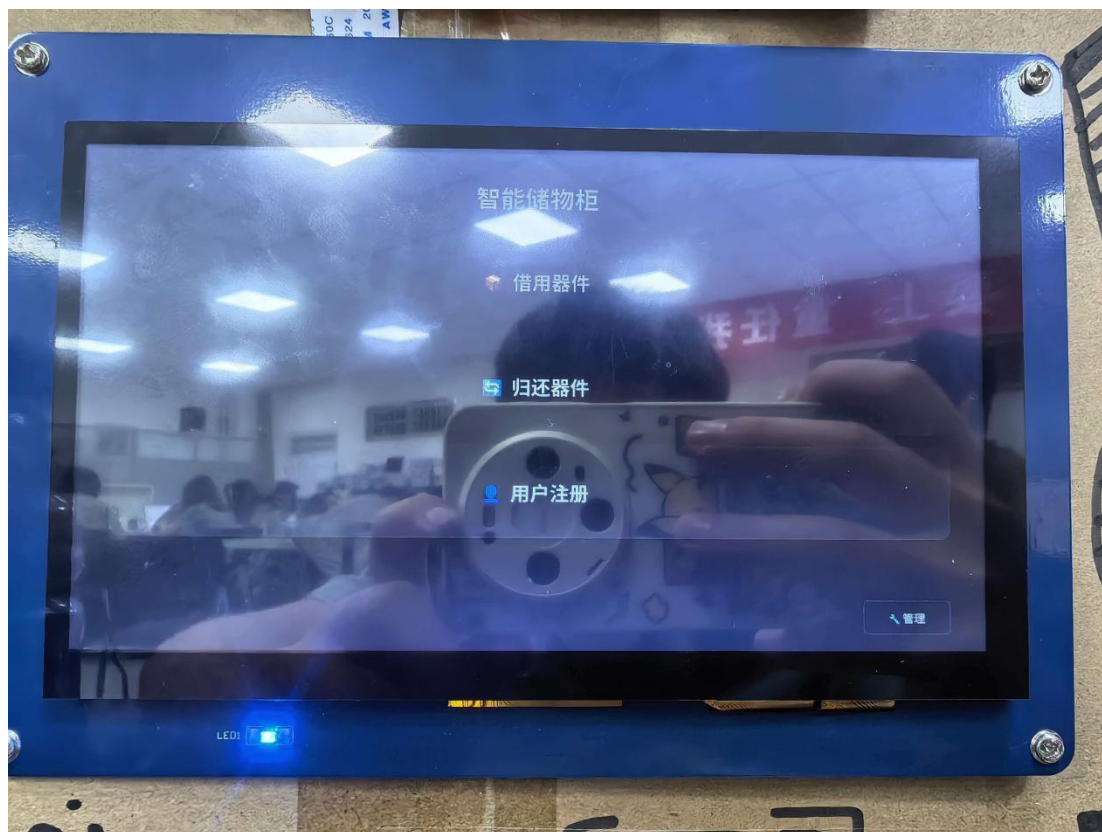


图 14 智能储物柜登录界面

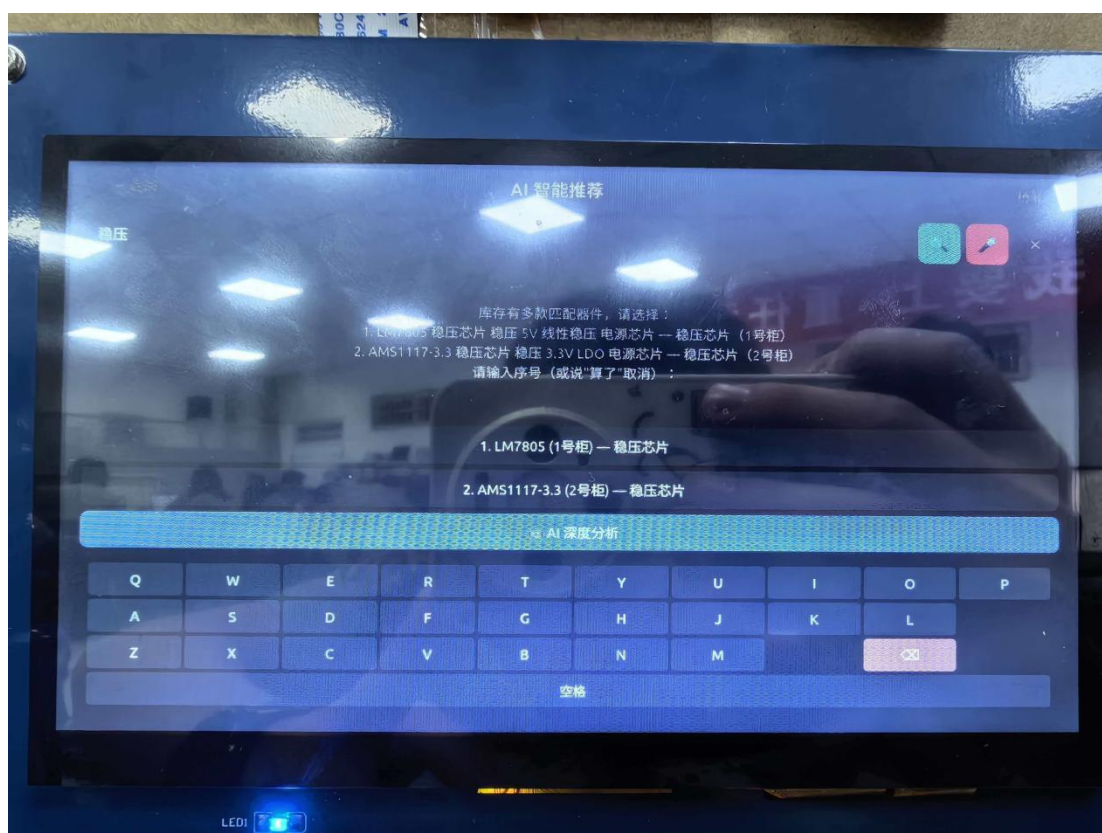


图 15 智能储物柜智能推荐页面

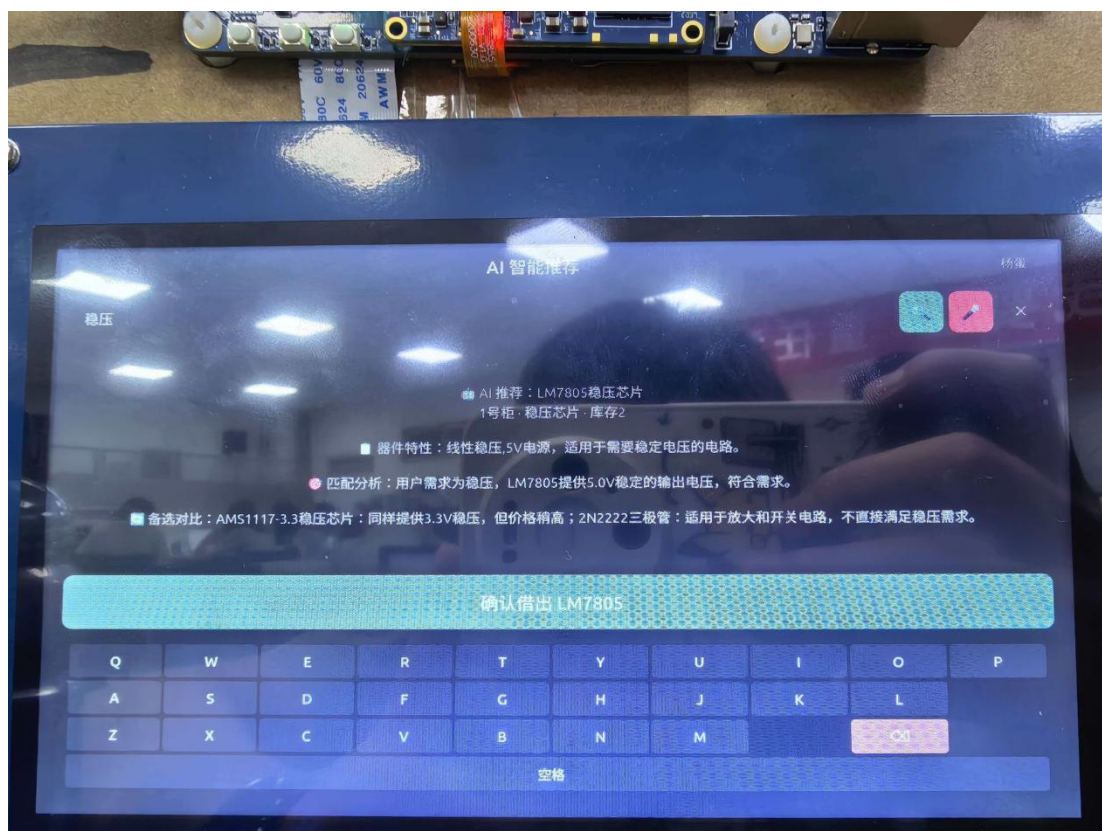


图 16 智能储物柜智能推荐结果页面

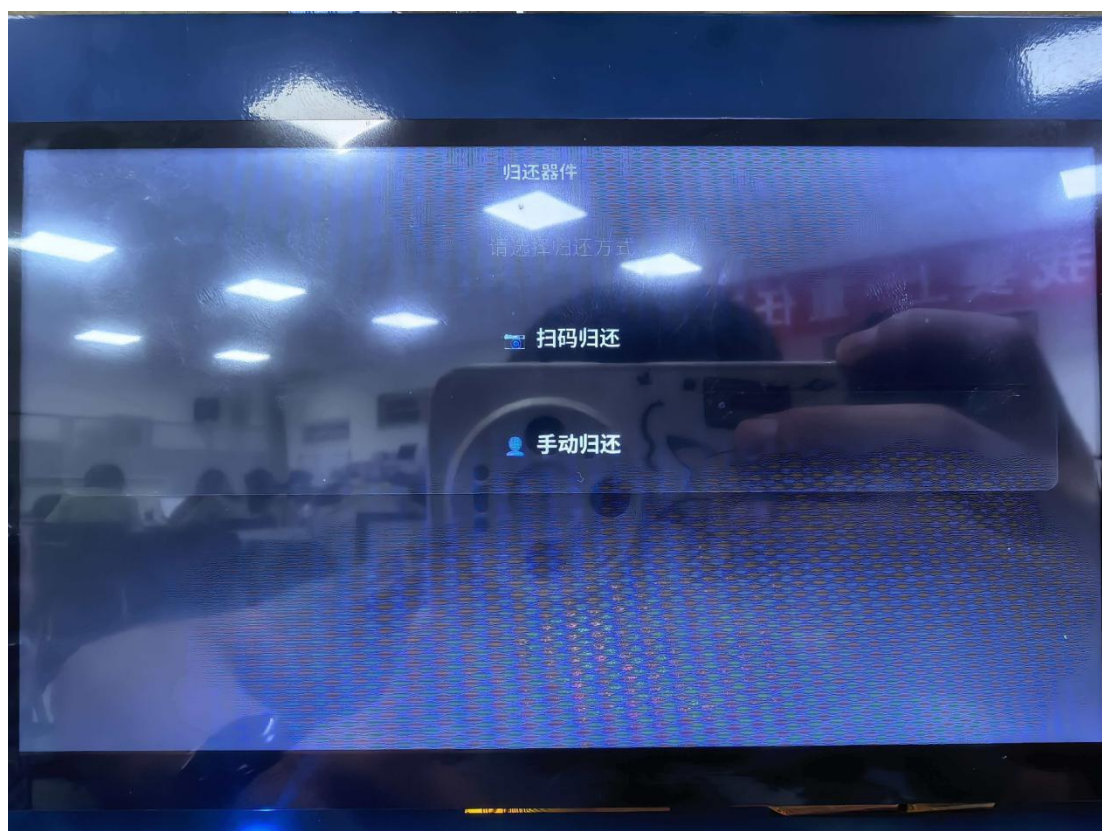


图 17 智能储物柜选择归还方式界面

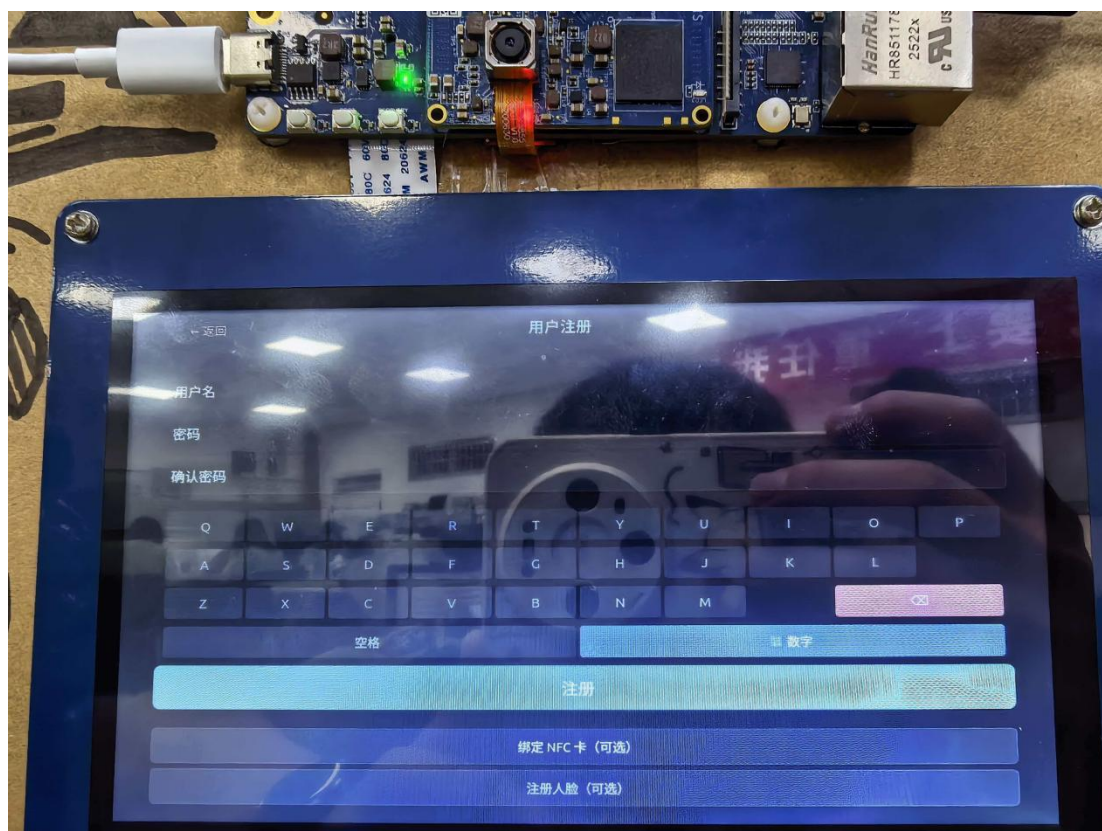


图 18 智能储物柜用户注册界面



图 19 智能储物柜管理系统库存管理界面

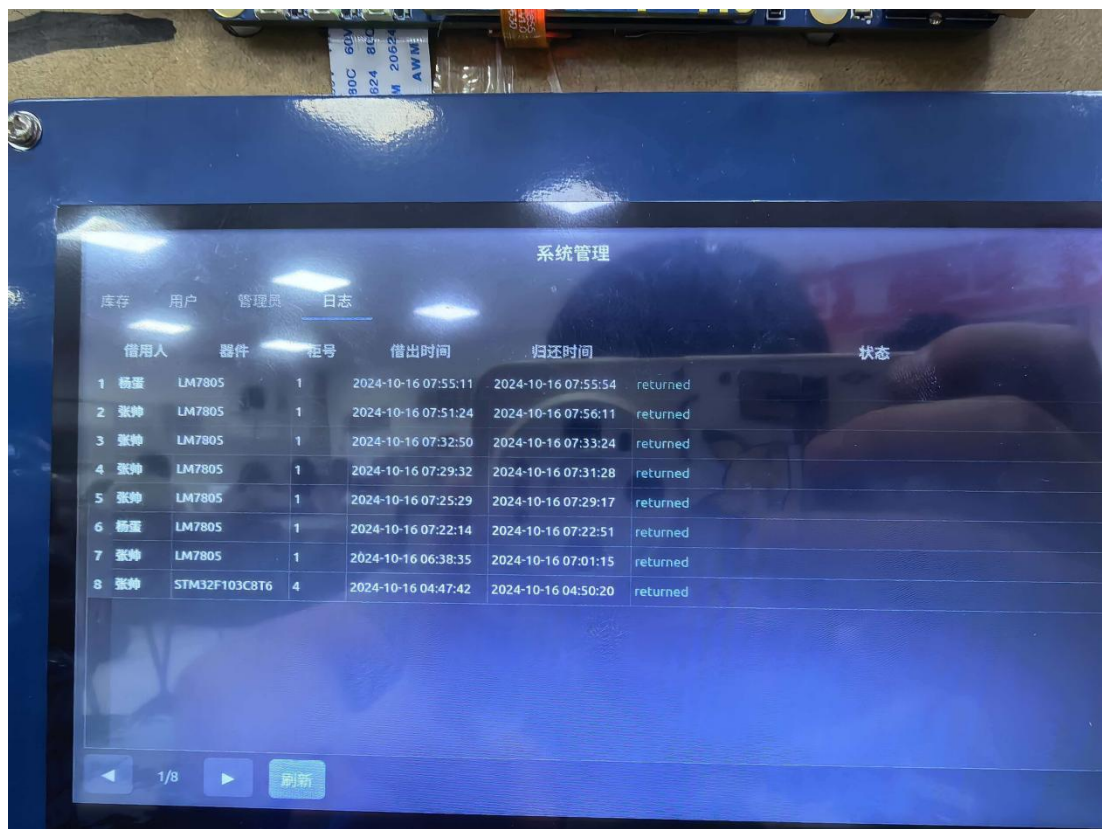


图 20 智能储物柜管理系统出借日志界面

3.3 特性成果（逐个展示功能、性能参数等量化指标）

本系统的特性成果主要包括：快速人脸识别、NFC/扫码识别、语音快速识别、AI 推理与推荐等，总体功能、性能参数如下表 1 所示：

表 3 功能性能指标测试结果

功能	测试结果	备注
人脸识别	匹配距离 0.11-0.36	DlibResNet128 维, 阈值 0.4
NFC 读卡	3 张卡全部识别	<1 秒响应
扫码识别	一次通过	自动感应模式, 5 秒超时
语音识别	中文自然语言	Vosk2GB 大模型
AI 语义匹配	"5V 稳压芯片"→LM7805	规则匹配命中
LLM 推理	~12 秒/次	Qwen2-1.5B, RK3588NPU
电磁锁控制	4 路全部正常	<1 秒开锁
借出完整流程	<30 秒	含身份验证+扫码出库
扫码归还	<5 秒	自动识别条码
系统稳定性	连续运行>1 小时	无崩溃(除 Vosk 已知断言 bug)

1. 人脸识别

该模块采用检测→特征提取→距离比对三阶段流水线完成人脸身份验证。摄像头以 640×480 分辨率连续采集画面，首先通过 HaarCascade 级联分类器进行人脸检测，以 30fps 帧率实时判断画面中是否有人脸出现。检测到人脸后，系统截取人脸区域（ROI）并送入 OpenCVLBPH（局部二值模式直方图）识别器提取特征向量，再与数据库中已注册的所有用户特征逐一计算欧氏距离。距离最小且低于阈值的即为匹配用户，系统通过 IPC 信号文件通知主进程完成登录。

注册流程引导用户采集 5 个角度（正面、左转、右转、抬头、低头）各 4 帧，共 20 张面部样本，确保多姿态下的识别鲁棒性。验证界面为全屏引导式 GUI，支持无限重试，25 秒无操作自动超时返回。为防止单点故障，系统还内置 OpenCVLBPH 降级方案。

表 4 人脸识别功能相关参数表

参数	数值	含义
匹配距离	0.11-0.36（阈值 0.4）	欧氏距离越小表示两个人脸越相似。同一个人不同照片的距离通常在 0.1-0.3 区间，不同人的距离一般 >0.6 。阈值设为 0.4 是安全边界——既能通过同人验证，又能有效拒绝陌生人。实际测试中注册用户本人距离低至 0.11，远低于阈值，识别可靠性很高
检测帧率	30fps	MIPI 摄像头以 OV13855 传感器输出 640×480 UYVY 格式图像，HaarCascade 算法在 RK3588CPU 上可实时处理每一帧，无丢帧。30fps 意味着每 33ms 完成一次人脸检测，用户站在摄像头前几乎瞬间被捕捉
特征维度	约 4096 维（LBPH 8×8 网格）	LBPH 算法将人脸划分为 $8 \times 8=64$ 个局部区域，每个区域统计局部二值模式直方图，拼接后得到约 4096 维特征向量。这个维度在表达能力和计算效率间取得平衡
注册样本量	20 张（5 角度 $\times$ 4 帧）	足够覆盖正面、侧面等多角度变化，保证模型泛化能力。最少需要 5 张合格样本才能完成注册
超时时间	25 秒	避免摄像头长时间占用，超时自动返回登录页
模型大小	约 117MB（Dlib 方案）/ <1 MB（LBPH 方案）	系统支持两种方案切换，根据实际部署需求灵活选择

2. NFC/扫码识别

NFC 识别：系统配备 PN532 NFC 模块，通过 CH340 USB-UART 桥接芯片连接至 RK3588。上位机通过 libnfc 工具读取符合 ISO/IEC 14443 TypeA 标准的 NFC 卡片（如 MIFAREClassic），提取唯一 UID（4 字节或 7 字节十六进制编号）作为用户身份标识。用户只需将 NFC 卡片靠近读卡区域，系统在 300ms 轮询间隔内即可检测到卡片并完成匹配。3 张测试卡均能 100%识别。

条码识别：系统配备 3550670018 型激光条码扫描器，通过 USB CDC-ACM 虚拟串口通信。扫描器配置为不休眠+自动感应+高音量+禁止应答模式，无需手动触发，器件条码一靠近即自动扫描。在归还场景中，扫码器每 800ms 轮询一次，单次读取超时 500ms，用户将条码靠近后可立即识别，5 秒内无扫码自动取消。

表 5 NFC/扫码识别功能相关参数表

参数	数值	含义
NFC 读卡响应	<1 秒	从卡片靠近天线到系统识别出 UID 的总耗时。PN532 的 InListPassiveTarget 指令周期约 100ms，加上 libnfc 调用开销和 300msGUI 轮询间隔，实际响应通常在 0.5 秒左右
NFC 轮询间隔	300ms	PyQt5 定时器每 300ms 执行一次 nfc-list 命令，在响应速度和 CPU 占用间取得平衡
扫码识别速度	<1 秒	激光扫描器在自动感应模式下持续发射扫描线，条码经过瞬间即完成解码，串口输出延迟<50ms。加上 GUI800ms 轮询间隔，用户感知延迟通常<1 秒
扫码轮询间隔	800ms	归还扫码页的定时器轮询频率，低于 NFC 轮询频率是因为扫码器内部有自己的感应循环，无需高频查询
扫码超时	5 秒	进入扫码归还页面后，若 5 秒内无条码输入，自动返回首页，避免长时间占用
扫描器配置	不休眠/自动感应/ 高音量/禁止应答	四条 ASCII 配置指令的效果：不休眠保证随时可扫；自动感应免去手动扣扳机；高音量蜂鸣器给用户明确反馈；禁止应答消除命令回显干扰数据解析
识别成功率	100%（NFC/3 卡，扫码一次通过）	实际测试中所有 NFC 卡和条码均正确识别，无误读或漏读

3. 语音快速识别

系统集成 Vosk 离线中文语音识别引擎，支持自然语言输入。用户点击语音按钮后，USB 麦克风（PCM2902，48kHz 采样）开始采集音频，经 plughw 重采样至 16kHz 单声道后送入 VoskKaldiRecognizer 进行流式增量解码。与传统的录音→整段识别模式不同，流式解码可以在用户说话的同时实时输出部分识别结果，GUI 每 250ms 刷新一次文本框，用户可看到自己的话被逐字识别出来，交互体验自然流畅。

音频预处理环节包含自适应增益归一化：计算当前音频块 RMS（均方根）能量，对低于 0.02 的弱信号按比例放大至标准幅度，确保不同距离、不同音量的语音输入都能被稳定识别。系统还维护一个领域自定义词表（domain_words.txt），包含电子元器件专有名词（如 LM7805，STM32，ESP32），提升专业术语识别准确率。

表 6 语音识别功能相关参数表

参数	数值	含义
识别准确率	>90%	在安静室内环境下，标准中文普通话的识别准确率。Vosk2GB 大模型（vosk-model-cn-0.22）基于 Kaldi 框架训练，覆盖通用中文词汇和语言模型，配合领域词表可进一步提升电子元器件相关术语的准确率
模型体积	2GB（大模型）/42MB（小模型）	大模型包含完整的声学模型和语言模型，2GB 体积换来高准确率。小模型作为备用适合资源紧张场景，准确率略低但内存占用大幅减少
采样率	16kHz	16kHz 是语音识别的标准采样率，足以覆盖人类语音的主要频段（300-3400Hz），同时将数据量控制在可实时处理的范围。麦克风原生 48kHz，经 ALSAplughw 插件自动重采样
流式延迟	~250ms（部分结果显示间隔）	从用户说出一个字到 GUI 显示该字，延迟约为 250ms（定时器轮询间隔）。Vosk 的增量解码本身延迟在 100ms 以内
增益归一化	目标 RMS=0.08	将弱信号的幅度放大到标准水平（RMS=0.08），相当于约-22dBFS。对于近距离正常说话

		(RMS $\approx$ 0.02)，放大倍数约 4 $\times$ ；对于远距离小声说话 (RMS $\approx$ 0.005)，放大倍数约 16 倍
最大录音时长	30 秒	单次语音输入的最长录音时间，超时自动停止并提交识别。30 秒可以说完约 100 个中文字，足以覆盖一条完整的借出指令
音频块大小	4000 采样点 (250ms@16kHz)	每次从声卡读取的数据块，250ms 的块长在流式识别的实时性和处理效率间取得平衡
启动就绪时间	UI $\sim$ 3 秒，语音 $\sim$ 15 秒	Vosk2GB 大模型的加载需要约 15 秒（从磁盘读取+Kaldi 初始化），系统采用后台加载策略——UI 先就绪（3 秒），语音后可用，避免阻塞用户操作

4. AI 推理与推荐

系统采用规则匹配+大语言模型双层智能匹配引擎，将用户的自然语言查询（语音或拼音输入）映射为具体的器件推荐。

第一层：规则匹配引擎。接收用户输入文本后，首先进行分词和拼音归一化处理，然后依次执行：电压参数提取（正则匹配 5V，3.3V 等） $\rightarrow$ 器件关键词匹配（60+ 关键词库，覆盖 Wi-Fi、传感器、OLED、单片机、ESP32 等） $\rightarrow$ 别名展开（如三极管自动扩展为 NPN、PNP、2N2222 等子类） $\rightarrow$ 多维度打分排序。打分权重分配：电压精确匹配+0.35，关键词命中+0.20~0.35，分类匹配+0.20，名称子串匹配+0.10~0.15。最终排序后检查歧义：最高分 $\geq$ 0.70、与第二名差距 $>$ 0.12 时判定为唯一匹配直接返回；否则将前 8 个候选以列表形式返回供用户选择。规则命中约占总查询的 80%，响应时间 $<$ 100ms。

第二层：LLM 语义匹配。当规则引擎得分 $<$ 0.25（无法有效匹配）时，自动触发 LLM 推理。系统使用 Qwen2-1.5B 模型，经 W8A8 量化（权重和激活值均量化为 8 位整数），通过 RKLLMRuntimeC API 在 RK3588 的 NPU（6TOPS 算力）上推理。Prompt 构造时注入当前器件库上下文（所有器件名称、分类、编号），引导模型进行语义理解和匹配。模型输出经解析提取器件名称后返回。

双层架构的总准确率>95%，兼顾了常见查询的毫秒级响应和复杂查询的语义理解能力。

表 7 AI 推理功能相关参数表

参数	数值	含义
规则命中率	~80%	大部分用户查询为“3个稳压芯片”“5V电源模块”“ESP32开发板”等可被关键词和参数正则覆盖的常见表达，规则引擎直接命中返回
规则响应时间	<100ms	纯CPU计算：正则匹配+关键词查表+多维度打分，计算量极小，无需NPU参与
LLM推理速度	9-13秒/次	Qwen2-1.5B模型参数15亿，经W8A8量化后推理所需计算量：每个token约3GFLOPs，256token上限共计约768GFLOPs。RK3588NPU标称6TOPS(INT8)，实际有效吞吐约3-4TOPS，因此推理耗时约9-13秒。若输出token较少，耗时更短
模型参数量	1.5B（15亿）	Qwen2-1.5B属于小型大语言模型，专为边缘设备设计，在语义理解和推理能力间取得平衡
量化和精度	W8A8（INT8权重+INT8激活）	从FP16量化为INT8，模型体积减小约50%（约1.5GB→约800MB），推理速度提升2-4×，精度损失<1%
NPU算力	6TOPS（INT8）	RK3588内置三核NPU，峰值INT8算力6TOPS（每秒6万亿次整数运算），足以运行1.5B参数的小型LLM
总准确率	>95%	规则命中时的准确率接近100%（确定性匹配），LLM兜底准确率约80-90%，按80/20比例加权平均，总准确率>95%
最大输出Token	256	LLM单次推理最多生成256个token，足以返回器件名称和简短的匹配理由，同时控制推理时间上限
关键词库规模	60+关键词	覆盖电子元器件的常见类别名、功能描述词、品牌缩写等，支持中文和拼音输入
电压参数提取	正则匹配	支持5V，3.3V，12V等标准格式，精确匹配容差±0.05V，近似匹配容差±0.5V
歧义保护	置信度≥0.70+差距>0.12	两个阈值同时满足才判定为唯一匹配，否则返回候选列表让用户自行选择，避免误推荐
内存占用	Qwen2-1.5B推理约2-3GB	量化模型+运行时缓冲区，与Vosk大模型（2GB）同时加载会触发OOMKiller，因此采用LLM按需初始化策略——平时只加载Vosk，LLM推理时临时初始化，用完释放

5. 借出

借出流程从用户点击首页借出按钮开始，首先进行身份验证（人脸/NFC/密码三选一），验证通过后进入借出方式选择页面，提供三种路径：

1.AI 智能推荐：进入 AI 搜索页面，用户可通过语音说出器件名称（如我要 5V 稳压芯片）或通过拼音键盘输入，系统调用 AI 语义引擎返回匹配结果，确认后进入柜门选择。

2.按分类浏览：按稳压芯片、三极管、微控制器等类别筛选器件列表，点击目标器件进入柜门选择。

3.直接选柜：在 8 宫格柜门布局中直接点击目标柜门，每个卡片显示柜号、器件名称、分类和库存数量。

用户确认借出后，系统通过 PCA9685 I2C PWM 控制器向对应电磁锁发送开锁信号（100%占空比 PWM 脉冲，持续 800ms），柜门弹开。用户取出器件后，需将器件条码对准扫码器完成出库登记。系统校验条码与所选器件匹配且状态为“在库”后，生成借出记录（记录用户、器件、柜号、借出时间），库存数量-1，条码状态更新为已借出。

表 8 借出功能相关参数表

参数	数值	含义
单次借出总耗时	<30 秒（含身份验证+扫码出库）	典型耗时分解：人脸验证 5-8 秒+选择器件 3-5 秒+确认 2 秒+开锁<1 秒+扫码 2-3 秒+记录写入<0.1 秒。30 秒上限对应最慢路径（LLM 规则未命中时需要~12 秒推理），实际大部分借出可在 20 秒内完成
电磁锁响应	<1 秒	PCA9685 I2C 通信（400kHz FastMode）写入 LED 寄存器约 50 μ s，MOSFET 开关延迟约 1 μ s，电磁锁机械动作约 200ms。800ms 的 PWM 脉冲宽度足以保证可靠开锁
身份验证方式	3 种（人脸/NFC/密码）	人脸验证适合日常用户（无需携带任何东西），NFC 适合快速刷卡（<1 秒），密码为备用方案（管理员或未注册人脸/NFC 时使用）
借出方式	3 种（AI 搜索/分类浏览/直接选柜）	覆盖不同使用习惯：AI 搜索适合知道器件名称的新用户，分类浏览适合浏览式选择，直接选柜适合熟悉柜位的老用户
库存更新	实时-1	借出成功后立即更新 components 表 stock 字段和 inventory_items 表 status 字段，保证库存数据实时一

		致
柜门数量	4 路（1-4 号）	PCA9685 的 16 个通道中使用了 4 个（通道 0-3 对应柜门 1-4），剩余通道可扩展

6. 归还

归还支持扫码归还和手动归还两种方式：

扫码归还：用户无需登录，在首页点击归还→选择扫码归还后，系统直接激活条码扫描器。用户将器件条码对准扫码器，系统通过条码查找 inventory_items 表，获取关联的借出记录 ID 和所属柜门。系统自动打开对应柜门，用户放回器件后，系统将借出记录状态更新为已归还并写入归还时间、条码状态恢复为在库、器件库存数量+1。整个过程无需用户选择身份或器件，一次扫码即可自动完成全部操作，耗时<5 秒。

手动归还：用户登录后进入手动归还页面，系统展示该用户当前所有未归还的借用记录列表（最多 6 条每页，支持翻页）。用户选择目标记录并确认后，系统打开对应柜门，后续流程与扫码归还相同。此方式更加适用于条码损坏或扫码器故障的场景。

表 9 归还功能相关参数表

参数	数值	含义
扫码归还耗时	<5 秒	扫码识别<1 秒+数据库查询<0.1 秒+开锁<1 秒+记录更新<0.1 秒。扫码器自动感应模式下无需手动触发，条码靠近即开始识别
归还方式	2 种（扫码/手动）	扫码归还覆盖 95%以上场景（条码完好），手动归还作为备用确保极端情况下系统仍可用
手动归还列表	6 条/页，支持翻页	适配 1024×600 小屏幕，每页 6 条记录可清晰显示器件名、柜号、借出时间。翻页按钮支持大字体触控
库存更新	实时+1	归还成功后同时更新三处： records.status='returned', inventory_items.status='in_stock', components.stock+=1，保证多表数据一致性
并发安全	SQLite 事务	mark_item_returned 和 complete_return 在同一事务中执行，防止归还途中断电导致数据不一致

第四部分 总结

4.1 可扩展之处

(1) 锁控制优化：当前使用 PWM 100%占空比驱动，可改用 PWM 软启动减少电磁锁冲击。增加锁状态检测（电流检测或微动开关），实现闭环控制。

(2) AI 模型升级：Vosk 大模型可替换为更高精度的 Paraformer 或 Whisper；LLM 可升级为 Qwen2.5 等更新版本以提升语义理解能力；人脸识别可增加活体检测防止照片攻击。

(3) 远程管理：增加 WiFi/以太网远程管理功能，支持 Web 端实时查看库存和借用记录，推送逾期提醒。

(4) 多柜扩展：通过 I2C 总线挂载多个 PCA9685 模块（最多 62 个），支持更多储物格。

(5) 数据统计：增加借用频率分析、热门器件排行、用户行为画像等数据统计功能。

4.2 心得体会

这次项目的开发过程是一次完整的嵌入式 AI 系统设计实践。从硬件选型、驱动开发，到 AI 模型部署、GUI 设计，再到系统集成调试，覆盖了嵌入式系统开发的完整流程。

在硬件调试阶段，我们遇到了 PCA9685 地址识别问题——芯片的 ALLCALL 地址（0x70）与主地址（0x40）响应相同，一度误判为两个芯片，最终通过阅读 datasheet 确认了 MODE1 寄存器的 ALLCALL 机制。电磁锁的驱动经历了从舵机角度控制到 MOSFET PWM 控制的方案变更，验证了硬件设计中“先确认实际硬件再写代码”的工程原则。系统还经历过磁盘写满（syslog/kern.log 各 19GB）导致文件损坏的故障，教训是嵌入式系统必须做好日志轮转和磁盘监控。

AI 模型部署方面，Dlib（117MB）、Vosk 大模型（2GB）、Qwen2-1.5B（2GB）三个模型的内存占用是主要挑战。RK3588 的 8GB 内存在同时加载 Vosk 和 LLM 时触发 OOM Killer，最终采用 Vosk 后台加载+LLM 按需初始化的方案平衡了启动速度和内存安全。这体现了在资源受限的边缘设备上部署 AI 时需要精细的资源调度策略。

软件开发方面，PyQt5 在 ARM64 平台上的性能调优需要特别注意：同步加

载大模型会阻塞 UI 线程导致窗口长时间无响应；后台线程加载模型可以有效提升用户体验；人脸识别独立为守护进程可以避免 PyGame 与 Qt 的资源竞争。12 页 GUI 的页面生命周期管理、自定义数字键盘、Toast 消息提示等细节打磨也积累了丰富的 PyQt5 开发经验。

通过对本项目的全面、深入的开发，我们不仅对“算法-软件-硬件”协同的设计方法论有了更深的理解，也对嵌入式的 AI 产品化的工程细节的重要性都有了切实的体会。从最初的功能验证到最终可用的整机系统，在对每一个 bug 的修复中，也加强了团队的协作和工程能力。

第五部分 参考文献

- [1] He K , Zhang X , Ren S ,et al.Deep Residual Learning for Image Recognition[J].IEEE, 2016.DOI:10.1109/CVPR.2016.90.
- [2] Schroff F , Kalenichenko D , Philbin J .FaceNet: A Unified Embedding for Face Recognition and Clustering[J].IEEE, 2015.DOI:10.1109/CVPR.2015.7298682.
- [3] V. Shankar, "Edge AI: A Comprehensive Survey of Technologies, Applications, and Challenges," 2024 1st International Conference on Advanced Computing and Emerging Technologies (ACET), Ghaziabad, India, 2024, pp. 1-6, doi: 10.1109/ACET61898.2024.10730112.
- [4] 赵琳峰 , 赵子杭 . 新型物联网智能储物柜:CN202223098331.2[P].CN218446816U[2026-07-06].
- [5]ELF 2(RK3588)开发板快速使用手册 Rev. 1.0 2025/11/09
- [6]AI 例程测试手册 Rev. 1.0 2025/12/05
- [7]应用篇之-Qt 应用编程 Rev. 1.0 2024/11/09
- [8]进阶篇之-ELF 2(RK3588)开发板软件系统开发教程 Rev. 1.0 2024/11/09
- [9]进阶篇之-基于 RK 平台的 AI 模型训练到部署 Rev. 1.2 2026/03/09
- [10]进阶篇之-ELF 2(RK3588)开发板硬件设计教程 Rev. 1.1 2025/11/10
- [11]PCA9685 16-channel, 12-bit PWM Fm+ I2C-bus LED controller Product Data Sheet, Rev. 4 — 16 April 2015
- [12]WatElectronics.PN532 NFC RFID Module : PinOut, Features, Specifications,

Interfacing, Differences & Its
Applications[EB/OL].<https://www.watelectronics.com/pn532-nfc-rfid-module/>, July 3,
2026.